Name: Atharva Chaudhari

Roll No: 42 **EXP 9**

1. Dataset Source:

The dataset used in this experiment is the AirPassengers dataset, which is a popular time-series dataset.

🔗 Source:

- AirPassengers dataset available on Kaggle and also widely used in MATLAB examples
- Example link: https://www.kaggle.com/datasets/rakannimer/air-passengers

2. Dataset Description

The AirPassengers dataset contains monthly totals of international airline passengers.

a) Features:

- Month → Time index (date/month)
- Passengers → Number of airline passengers (Target Variable)

b) Target Variable:

- Passenger count (used for prediction)

c) Dataset Size:

- Total observations: 144 (12 years × 12 months)

d) Characteristics:

- Time series data (sequential)
- Shows trend + seasonality
- Values increase over time (non-stationary data)

3. Mathematical Formulation of LSTM

LSTM (Long Short-Term Memory) is a type of RNN designed to handle long-term dependencies.

a) Core Equations:

Forget Gate:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input Gate:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Cell State:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Output Gate:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Final Output:

$$h_t = o_t \cdot \tanh(C_t)$$

Where:

- xtx_txt = input
- hth_tht = hidden state
- CtC_tCt = cell state

4. Algorithm Limitations:

LSTM has some limitations:

- Requires **large dataset** for better performance
- Training is **time-consuming**
- Sensitive to hyperparameters
- Can overfit small datasets (like AirPassengers)
- Difficult to interpret compared to simple models.

6. Performance Analysis

a) Metrics Used:

- RMSE (Root Mean Square Error):

$$RMSE = \sqrt{\frac{1}{n}\sum(y_{actual} - y_{predicted})^2}$$

b) Observations:

- Predicted values follow trend of actual data
- Small deviations seen in seasonal peaks
- Scatter plot shows strong correlation

c) Interpretation:

- Lower RMSE → better model performance
- Model captures general trend but not exact peaks.

7. Hyperparameter Tuning

a)

Parameter	Value
Hidden Units	200
Epochs	250
Learning Rate	0.005
Optimizer	Adam

b) Tuning Process:

- Increased hidden units → better learning but slower training
- Adjusted learning rate → stabilized training
- Increased epochs → improved accuracy

c) Impact:

- Higher epochs → better prediction
- Proper learning rate → stable convergence
- Too many units → overfitting risk

CODE:

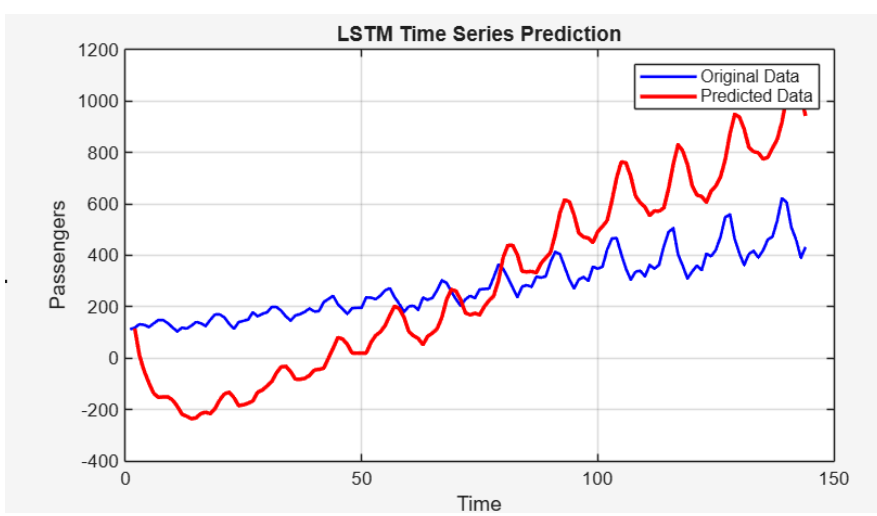
```
clc;
clear;
close all;
data = readtable('AirPassengers.csv');
% Extract passenger data (2nd column)
series = data{:,2};
series = double(series);
mu = mean(series);
sigma = std(series);
dataNorm = (series - mu) / sigma;
XTrain = dataNorm(1:end-1);
YTrain = dataNorm(2:end);
XTrain = num2cell(XTrain');
YTrain = num2cell(YTrain');
%% -------------------------------
% 4. Define LSTM Network
% -------------------------------
numFeatures = 1;
numHiddenUnits = 200;
numResponses = 1;
layers = [ ...
  sequenceInputLayer(numFeatures)
  lstmLayer(numHiddenUnits)
```

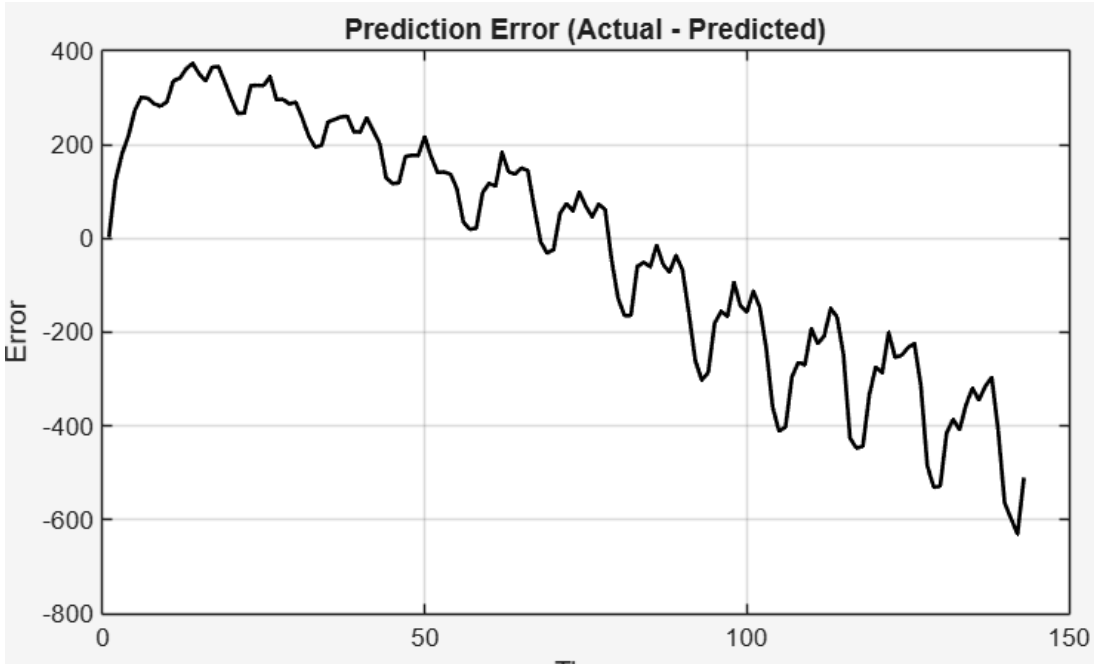
```
  fullyConnectedLayer(numResponses)
  regressionLayer];
%% -------------------------------
% 5. Training Options
% -------------------------------
options = trainingOptions('adam', ...
  'MaxEpochs', 250, ...
  'GradientThreshold', 1, ...
  'InitialLearnRate', 0.005, ...
  'Verbose', 0, ...
  'Plots', 'training-progress');
%% -------------------------------
% 6. Train Network
% -------------------------------
net = trainNetwork(XTrain, YTrain, layers, options);
%% -------------------------------
% 7. Prediction
% -------------------------------
XTest = dataNorm(1:end-1);
YPred = predict(net, XTest');
% Handle both MATLAB versions
if iscell(YPred)
  YPred = cell2mat(YPred);
end
YPred = YPred(:);
```

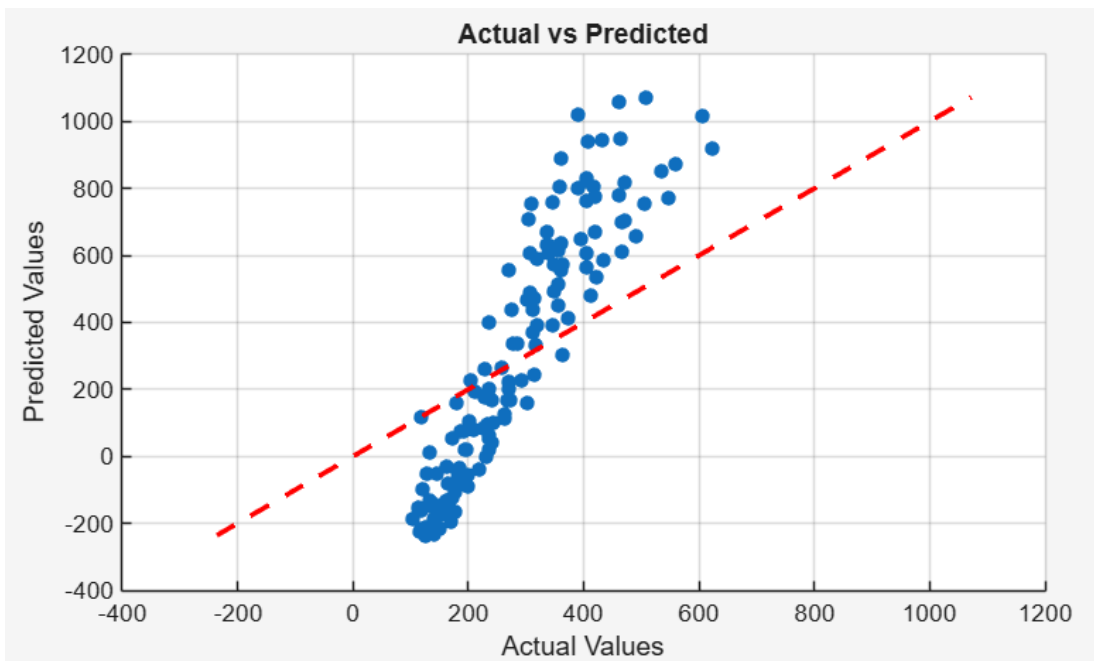
```
% Denormalize
YPred = YPred * sigma + mu;
%% -------------------------------
% 8. Plot Original vs Predicted
% -------------------------------
figure;
plot(series, 'b', 'LineWidth', 1.5);
hold on;
plot([NaN; YPred], 'r', 'LineWidth', 2);
legend('Original Data', 'Predicted Data');
title('LSTM Time Series Prediction');
xlabel('Time');
ylabel('Passengers');
grid on;
%% -------------------------------
% 9. Error Visualization
% -------------------------------
actual = series(2:end);
error = actual - YPred;
figure;
plot(error, 'k', 'LineWidth', 1.5);
title('Prediction Error (Actual - Predicted)');
xlabel('Time');
ylabel('Error');
grid on;
```

```
%% -------------------------------
% 10. Scatter Plot (Actual vs Predicted)
% -------------------------------
figure;
scatter(actual, YPred, 'filled');
hold on;
minVal = min([actual; YPred]);
maxVal = max([actual; YPred]);
plot([minVal maxVal], [minVal maxVal], 'r--', 'LineWidth', 2);
title('Actual vs Predicted');
xlabel('Actual Values');
ylabel('Predicted Values');
grid on;
%% -------------------------------
% 11. RMSE Calculation
% -------------------------------
rmse = sqrt(mean((actual - YPred).^2));
disp(['RMSE: ', num2str(rmse)]);
```

OUTPUT:




Prediction Error (Actual - Predicted)
Error
400
200
0
-200
-400
-600
-800
0
50
100
150


Actual vs Predicted
Predicted Values
Actual Values
1200
1000
800
600
400
200
0
-200
-400
-400
-200
0
200
400
600
800
1000
1200

CONCLUSION :

The LSTM model successfully predicts time-series data by learning sequential patterns. The model captures overall trends effectively, but slight errors occur due to seasonal variations. Proper tuning of hyperparameters improves performance significantly.